

Text as Data (TaD) -- Comprehensive A-Grade Exam Notes

University of Glasgow | COMPSCI 5096/5106 | 2025-2026 Exam Format: Open book, 1h 30m, Answer ALL 3 questions, ~60 marks total

Table of Contents

1. [Introduction, Tokens & Document Similarity](#)
 2. [Geometric Similarity, Text Distributions & Clustering](#)
 3. [Language Modelling](#)
 4. [Text Classification](#)
 5. [Contextual Word Embeddings](#)
 6. [POS Tagging & Parsing](#)
 7. [Ethics & Information Extraction](#)
 8. [Information Extraction \(Advanced\)](#)
 9. [Large Language Models](#)
 10. [Exam Strategy & Past Paper Analysis](#)
 11. [Crib Sheet Reference \(All Formulas\)](#)
-

1. Introduction, Tokens & Document Similarity

1.1 The NLP Pipeline

The standard NLP pipeline for processing text consists of these ordered steps:

1. **Data Cleaning** -- removing non-relevant content (HTML tags, links, tables, boilerplate)
2. **Tokenization** -- splitting text into tokens (words, subwords, or characters); may include sentence splitting
3. **Normalization** -- reducing variation in tokens
4. **Stemming**: rule-based suffix stripping (e.g., "running" -> "run", "textiles" -> "textil"). Faster but cruder.
5. **Lemmatization**: dictionary-based reduction to base form (e.g., "better" -> "good"). More accurate but slower.
6. **Stopword Removal** -- removing high-frequency, low-information words (the, is, a, of)

Exam Tip: When asked to justify preprocessing steps, always give a *domain-specific example*. E.g., for patent classification: case folding ensures "Textile" (start of sentence) and "textile" are treated identically; stemming ensures "textiles" and "textile" map to the same token.

1.2 Encodings

Encoding	Bytes per Character	Characters
ASCII	1 byte	128 characters (English letters, digits, symbols)
Unicode	2-4 bytes	150,000+ characters (all world scripts)

1.3 Tokenization Approaches

- **Whitespace tokenization**: split on spaces
- **Rule-based**: regex patterns for punctuation, contractions
- **Subword tokenization** (BPE, WordPiece): see Section 5

1.4 Similarity Measures (Set-Based)

Given two documents as sets of tokens X and Y:

Measure	Formula	Range
Overlap count	$ X \cap Y $	$[0, \min(X , Y)]$
Overlap Coefficient	$ X \cap Y / \min(X , Y)$	$[0, 1]$
Sorensen-Dice Coefficient	$2 X \cap Y / (X + Y)$	$[0, 1]$
Jaccard Similarity	$ X \cap Y / X \cup Y $	$[0, 1]$

Key relationship: $\text{similarity}(X, Y)$ is proportional to $1 - \text{distance}(X, Y)$

1.5 Properties of a Metric (Distance Function)

A distance function d is a **metric** if it satisfies ALL four:

1. **Identity:** $d(X, X) = 0$
2. **Positivity:** $d(X, Y) > 0$ where $X \neq Y$
3. **Symmetry:** $d(X, Y) = d(Y, X)$
4. **Triangle Inequality:** $d(X, Z) \leq d(X, Y) + d(Y, Z)$

A **semi-metric** violates triangle inequality. A **pseudo-metric** violates positivity.

Exam Tip (2022 Q2d): Sorensen-Dice is a semi-metric -- it breaks triangle inequality. To show this: find three sets A, B, C where $\text{dice}(A, B) > \text{dice}(A, C) + \text{dice}(B, C)$. Convert similarity to distance first: $d = 1 - \text{sim}$.

1.6 Worked Example: Set Similarity Calculation

Probe A: {a, eda, bcda, bcda, bce} (5 unique tokens) **Probe B:** {ca, eda, bcba, bcda, bce} (5 unique tokens) **Intersection:** {eda, bcda, bce} = 3 **Union:** {a, eda, bcda, bcda, bce, ca, bcba} = 7

Sorensen-Dice = $2 * 3 / (5 + 5) = 6/10 = 0.6$
Jaccard = $3 / 7 = 0.429$

1.7 Regular Expressions Quick Reference

Pattern	Meaning
.	Any character
[abc]	One of a, b, c
[^abc]	Any char except a, b, c
[a-z]	Range
\d, \w, \s	Digit, word char, whitespace
\D, \W, \S	Negated versions
*, +, ?	0+, 1+, 0 or 1
^, \$	Start/end of string
\b	Word boundary

1.8 N-grams

An **n-gram** is a contiguous sequence of n items from text.

Theoretical maximum n-grams for alphabet of size A and text of length L: - If $A^n \leq L - n + 1$: max = A^n - If $A^n > L - n + 1$: max = $L - n + 1$

Example (2022 exam): Alphabet = 5 chars, text length = 593:

n	Max n-grams
1	5 (= 5^1)
2	25 (= 5^2)
3	125 (= 5^3)
4	590 (= $593-4+1$, since $5^4=625 > 590$)
5	589 (= $593-5+1$, since $5^5=3125 > 589$)

2. Geometric Similarity, Text Distributions & Clustering

2.1 Term Frequency Variants

Method	Formula
Raw TF	$tf(t,d) = \text{count of } t \text{ in } d$
Log-transformed TF	$1 + \log_{10}(tf(t,d))$ if $tf > 0$, else 0
IDF	$\log(N / df(t))$ where N = total docs, df = docs containing t
TF-IDF	$(1 + \log(tf(t,d))) * \log(N / df(t))$
Linear TF-IDF	$tf(t,d) * idf(t)$

Zipf's Law: A few words are very frequent; many words are very rare. The frequency of a word is inversely proportional to its rank.

2.2 Vector Similarity Measures

Measure	Formula	Notes
Euclidean Distance	$\sqrt{\sum (x_i - y_i)^2}$	Sensitive to vector magnitude
Cosine Similarity	$(d1 \cdot d2) / (\ d1\ * \ d2\)$	Magnitude-independent; preferred for text

Exam Tip: Cosine similarity is preferred for text because documents of different lengths can still be compared meaningfully -- it measures angle, not magnitude.

2.3 TF-IDF Worked Example (2025 Resit Q1)

Given vocabulary with IDF values and document DocX: G C G A H A G :

Token	TF	IDF	TF-IDF
A	2	2	4
B	0	4	0
C	1	1	1
G	3	1	3
H	1	3	3

Dense representation: [4, 0, 1, 0, 0, 0, 3, 3, 0, 0]

Reconstructing from TF-IDF: Divide TF-IDF value by IDF to get TF. Note: **word order is lost** in bag-of-words -- this is a key ambiguity.

2.4 K-Means Clustering

Algorithm: 1. **Step 0:** Initialize k centroids (randomly or via Forgy/Random Partition) 2. **Step 1:** Assign each data point to its nearest centroid 3. **Step 2:** Recalculate centroids as the mean of assigned points 4. **Repeat** Steps 1-2 until convergence (assignments stop changing)

Centroid Initialization Methods:

Method	Description
Forgy	Randomly select k data points as initial centroids
Random Partition	Randomly assign each point to a cluster, then compute centroids

Choosing k: Use the **Elbow Method** -- plot total within-cluster distance vs. k, pick the "elbow" point where improvement diminishes.

2.5 Measuring Clustering Quality

Intrinsic Measures (no external labels needed): - **Cohesion(i):** average distance from point i to other points in same cluster (lower = better) - **Separation(i):** minimum average distance from point i to points in another cluster (higher = better)

Silhouette Score:

```
silhouette(i) =
  1 - cohesion(i)/separation(i)    if cohesion < separation
  0                                if cohesion = separation
  separation(i)/cohesion(i) - 1    if cohesion > separation
```

Range: [-1, 1]. Higher = better clustering.

Extrinsic Measures (require external labels): - **Purity:** fraction of each cluster that belongs to the majority class - **Human evaluation**

3. Language Modelling

3.1 Probability Foundations

- $0 \leq P(x) \leq 1$ and sum of all $P(x) = 1$
- **Bayes' Rule:** $P(A|B) = P(B|A) * P(A) / P(B)$
- **Chain Rule:** $P(A,B,C) = P(A|B,C) * P(B|C) * P(C)$
- **Independence:** A and B are independent iff $P(A|B) = P(A)$ and $P(B|A) = P(B)$

3.2 Language Model Goal

Estimate $P(w_1, w_2, \dots, w_n)$ or $P(w_{k+1} | w_1, \dots, w_k)$

Counting approach:

$$P(w_1, w_2, \dots, w_n) = \text{Count}(w_1, w_2, \dots, w_n) / N$$

3.3 Markov Assumption and N-gram Models

The **Markov assumption** states that the probability of a word depends only on the previous $n-1$ words:

Model	Assumption	Formula
Unigram	$P(w_i)$ independent	$\text{Count}(w_i) / N$
Bigram	Depends on previous word	$\text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$
Trigram	Depends on previous 2 words	$\text{Count}(w_{i-2}, w_{i-1}, w_i) / \text{Count}(w_{i-2}, w_{i-1})$

3.4 Smoothing

Problem: Unseen n-grams get probability 0, which breaks the model.

Add-k Smoothing:

$$P_{\text{add-k}}(t | \text{context}) = (\text{count}(t, \text{context}) + k) / (\text{sum over } t' \text{ of } \text{count}(t', \text{context}) + k * |V|)$$

where $|V|$ is the vocabulary size. When $k=1$, this is Laplace smoothing.

Interpolation:

$$P_{\text{interpolated}}(t_i | t_{i-1}, t_{i-2}) = l_1 * P(t_i | t_{i-1}, t_{i-2}) + l_2 * P(t_i | t_{i-1}) + l_3 * P(t_i)$$

where $l_1 + l_2 + l_3 = 1$

Jelinek-Mercer Smoothing:

$$P_{JM}(t \mid \text{doc}, \text{collection}) = \lambda * P(t \mid \text{doc}) + (1-\lambda) * P(t \mid \text{collection})$$

3.5 LM Evaluation

Metric	Formula	Interpretation
Entropy	$-\sum(P(x) * \log_2(P(x)))$	Uncertainty in distribution
Cross-Entropy	$-(1/n) * \sum(\log_2(P(w_i)))$	How well model predicts test data
Perplexity	$2^{(\text{Cross-Entropy})}$ or $(\prod P(w_i))^{(-1/n)}$	Lower = better model

Evaluation types: - **Intrinsic:** evaluate LM directly (perplexity) - **Extrinsic:** evaluate LM as part of another task (e.g., machine translation quality) - **Syntactic:** does generated text match language rules? - **Semantic:** does generated text convey actual meaning?

3.6 Text Generation Strategies

Strategy	Description
Greedy	Always pick the highest-probability next token (deterministic, boring)
Sampling	Sample from the probability distribution (more diverse)
Beam Search	Maintain k best candidate sequences, expand each, prune to top k
Top-k	Sample from top k highest-probability tokens only
Top-p (nucleus)	Sample from smallest set of tokens whose cumulative probability $\geq p$
Contrastive Search	Balance probability and diversity from previous context

3.7 Worked Example: N-gram Generation (2024 Q2)

Text: BCBBCBBABCCCBCCBCCB (20 chars, alphabet: {A, B, C})

Unigram counts: B=10, C=8, A=2. Total=20. **Greedy unigram:** Always picks B -> generates BBB

Trigram greedy: Look at last 2 chars CB, count all trigrams starting with CB : - CBB: 3, CBC: 1 -> pick B - Then context is BB : BBA: 1, BBB: 0, BBC: 1 -> tie, pick first alphabetically: A - Continue with BA : BAB: 1 -> pick B - Result: BAB

Bigram beam search (width 2): Track 2 best partial sequences at each step, expanding all possibilities and keeping top 2 by cumulative probability.

Exam Tip: For generation questions, ALWAYS show your n-gram counts explicitly before generating. Common mistake: forgetting to use the *newly generated* characters as context for subsequent tokens.

3.8 Zero-Probability Problem

A bigram model cannot generate a sequence if any constituent bigram has zero count in training data.

Solution: Apply **add-k smoothing** to give all unseen n-grams a small probability, then use **sampling** (not greedy) to allow selection of low-probability tokens.

4. Text Classification

4.1 Types of Classification

Type	Description	Example
Binary	2 classes	Spam vs. not spam
Multi-class	1 label from n classes	Sentiment: pos/neg/neutral
Multi-label	Multiple labels per instance	Recipe: Italian AND pasta AND comfort food

Exam Tip (2025 Q2a): If categories overlap (e.g., "Italian" and "pasta"), it is multi-label not multi-class. This distinction is frequently tested.

4.2 Classification Metrics

Binary Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Core Metrics:

Accuracy = $(TP + TN) / (TP + FP + TN + FN)$
 Precision = $TP / (TP + FP)$ -- "Of predicted positives, how many correct?"
 Recall = $TP / (TP + FN)$ -- "Of actual positives, how many found?"
 F1 = $2 * TP / (2 * TP + FP + FN) = 2 * (P * R) / (P + R)$
 F-beta = $(1 + \text{beta}^2) * (P * R) / (\text{beta}^2 * P + R)$

Multi-class Averaging:

Method	Description
Macro	Unweighted average of per-class metric
Weighted	Weighted average by class sample count
Micro	Sum TP/FP/TN/FN across all classes, then compute metric

4.3 ROC Curve and AUROC

- **ROC Curve:** Plot True Positive Rate (recall) vs False Positive Rate

- **Construction:** Start bottom-left. For each prediction (ordered by confidence): actual positive -> move up; actual negative -> move right
- **AUROC:** Area under ROC curve. 0.5 = random, 1.0 = perfect

4.4 Data Splitting and Cross-Validation

Approach	Use Case
Train/Val/Test split	Standard approach; train on train, tune on val, report on test
K-Fold Cross-Validation	For small datasets; split into k folds, rotate test fold

4.5 Common Classifier Problems

Problem	Description
Noise	Errors in labels or features
Missing features	Needed discriminative features not available
Underfitting	Model too simple; poor on train AND test
Overfitting	Model too complex; good on train, poor on test
Class imbalance	One class dominates; model biased toward majority
Test set leakage	Training data appears in test set; inflated metrics

4.6 Detecting Overfitting vs Underfitting (2022 Q3d)

Scenario	Train Performance	Test Performance	Diagnosis
System A	Slightly better than baseline*	Better than baseline*	Good generalization
System B	Much better than baseline*	Worse than baseline	Overfitting

*asterisk = statistically significant

Exam Tip: Always check if test improvements are *statistically significant*. System A's test precision is significant (*) but recall is not -- discuss both.

4.7 Dataset Construction Pitfalls (2025 Q2a)

When building a labelled dataset, watch for: 1. **Multi-label vs multi-class confusion** -- overlapping categories need multi-label 2. **Insufficient data** -- too few samples per class 3. **Automated labels** -- lower quality than human annotation 4. **Circular evaluation** -- using search to build labels means classifier can't outperform search 5. **Data duplication** -- causes class imbalance, test set leakage, unrealistic metrics

4.8 Anomaly Detection in Metrics (2025 Q2c)

Common anomalies to spot: - Perplexity reported for classification (wrong metric -- it's for language models) - Different number of classes in train vs test confusion matrices - Accuracy doesn't match confusion matrix values - Class distribution mismatch between train and test - Validation set larger than training set (against best practice) - Perfect test performance with poor train performance (suspicious) - F1 not between precision and recall (mathematically impossible) - Macro precision/recall reported for binary problems (just use precision/recall)

5. Contextual Word Embeddings

5.1 Static Word Embeddings

Distributional Semantics (IBM Model / SVD): - Represent each word as a vector of co-occurring context words - Window size k defines context: $[t_{i-k}, \dots, t_{i-1}, t_i, t_{i+1}, \dots, t_{i+k}]$ - Use dictionary of sparse vectors; single pass through corpus

Window size trade-off: - **Too large k :** contexts become too broad and noisy, vectors inconsistent - **Too small k :** not enough information to represent word meaning

Function word problem: Frequent words like "the", "of" appear in all contexts, so $P(\text{context}|\text{word})$ is near-uniform -- their vectors are unreliable and uninformative.

Word2Vec: - Learns dense vectors via noise contrastive estimation - **Negative samples** provide reference outputs, enabling learning from 2 classes (true context vs. random) instead of softmax over entire vocabulary - **Polysemy problem:** Word2Vec produces ONE vector per word. For polysemous words (e.g., "python" = snake/programming), nearest neighbors mix senses. This hurts downstream tasks.

Handles synonymy (similar words get similar vectors) but **NOT polysemy** (one word, multiple meanings).

5.2 Contextual Word Embeddings

Model	Type	Key Feature
ELMo	Bi-LSTM	Different vector per context
BERT	Encoder (Transformer)	Bidirectional, masked LM
GPT	Decoder (Transformer)	Unidirectional, causal LM

These produce **different vectors for the same word in different contexts**, solving polysemy.

5.3 Self-Attention Mechanism

```

Q = Input * W_Q    (queries -- "what am I looking for?")
K = Input * W_K    (keys -- "what do I contain?")
V = Input * W_V    (values -- "what information do I provide?")

Attention(Q, K, V) = softmax(Q * K^T / sqrt(d_k)) * V

```

- d_k = dimension of keys (scaling prevents large dot products)
- Multi-head attention:** run multiple attention heads in parallel, concatenate results

5.4 Subword Tokenization: BPE Algorithm

Training Phase: 1. Pre-tokenize text into words 2. Initialize vocabulary with all individual characters 3. Repeat until target vocabulary size: - Find the most common adjacent symbol pair - Merge all instances into a new symbol - Add new symbol to vocabulary

Encoding Phase (applying learned rules): 1. Split word into characters 2. Apply merge rules in order of learning

Worked Example (2024 Q1c):

Encoding `mississippi pie` with rules: `is, pi, ie, pie, ppi, sippi`

```
mississippi -> m i s s i s s i p p i
Rule 1 (i,s->is): m i s s i s s i p p i
Rule 2 (p,i->pi): m i s s i s s i p pi
                  (no more p,i pairs adjacent)
Rule 3 (i,e->ie): (no i,e pairs) -> same
Rule 4 (p,ie->pie): (no p,ie) -> same
Rule 5 (p,pi->ppi): m i s s i s s i ppi
Rule 6 (si,ppi->sippi): m i s s i s sippi

pie -> p i e
Rule 1: (no i,s) -> p i e
Rule 2: (p,i->pi): pi e
Rule 3: (i,e->ie): (no i,e left) -> pi e
Rule 4: (no p,ie) -> pi e
          Actually: pi + e, no merge applicable.
```

BPE Vocabulary Size Trade-offs (2024 Q1d): - **Too low (e.g., 100):** Most tokens are single characters, losing morphological information; sequences become very long, increasing computation - **Too high (e.g., 100M):** Approaches word-level tokenization; rare words stored as full tokens with poor embeddings; large embedding matrix wastes memory

5.5 Special Tokens

Token	Purpose
[CLS]	Classification token (sentence-level representation)
[SEP]	Separator between sentences
[PAD]	Padding for batch processing
[MASK]	Masked token for MLM training
[UNK]	Unknown token (less needed with BPE since chars cover Unicode)

Exam Tip (2024 Q1b): BPE operates on bytes, so it can represent ANY Unicode character. However, [UNK] may still be needed for corrupted/invalid byte sequences.

5.6 Transformer Training Tasks

Task	Description	Model
Causal LM	Predict next word (left-to-right)	GPT
Masked LM	Predict masked word (bidirectional)	BERT
Next Sentence Prediction	Does sentence B follow A?	BERT
Corrupted Token Detection	Spot the replaced word	ELECTRA

5.7 Encoder vs Decoder

	Encoder (BERT)	Decoder (GPT)
Attention	Bidirectional (sees all tokens)	Causal/masked (sees only left context)
Best for	Classification, NER, understanding	Generation, completion
Training	Masked LM	Causal LM

5.8 Transfer Learning: Pretraining + Fine-tuning

1. **Pretrain** on large unlabelled corpus (expensive, done once)
2. **Fine-tune** on small labelled task-specific data (cheap, done per task)

Exam Tip (2022 Q3e): To add POS information to BERT: create a new embedding layer (one embedding per POS tag), add it to the input embedding layer alongside token and positional embeddings. This differs from bag-of-features models where each word-POS pair is a separate sparse feature.

5.9 Stopwords in BoW vs Transformers (2024 Q1a)

- **Removed in BoW:** Stopwords add noise since BoW ignores word order. "The cat sat on the mat" and "mat the the cat on sat" are identical in BoW.
- **Kept in Transformers:** Transformers use positional encoding and attention to model word relationships. Stopwords carry syntactic meaning. "Not good" vs "good" -- removing "not" changes sentiment entirely.

6. POS Tagging & Parsing

6.1 Hidden Markov Model (HMM) for POS Tagging

Two probability components: - **Transition probabilities:** $P(y_i | y_{i-1})$ -- probability of tag given previous tag - **Emission probabilities:** $P(x_i | y_i)$ -- probability of word given its tag

6.2 Decoding Strategies

Strategy	Description	Quality
Greedy	Pick best transition/emission at each token independently	Suboptimal; fast
Viterbi	Dynamic programming; finds globally optimal path	Optimal; slower

6.3 Domain Mismatch Problem (2022 Q3c)

An off-the-shelf POS tagger trained on news text may perform poorly on domain-specific text (e.g., patents, medical) because: - Specialized vocabulary not in training data - Different sentence structures

Solutions: 1. **Fine-tune** the tagger on domain-specific annotated data 2. **Pre/post-processing** with domain-specific dictionaries for known domain words

6.4 Constituency Parsing

Breaks sentences into **nested constituents**: - **NP** (Noun Phrase): "the big cat" - **VP** (Verb Phrase): "sat on the mat" - **PP** (Prepositional Phrase): "on the mat"

6.5 Dependency Parsing

Creates a **directed graph** of word relationships: - **Properties:** weakly connected, acyclic, single root - Each word has exactly one head (except root) - Relations: subject, object, modifier, etc.

Transition-based Dependency Parsing:

Action	Description
Shift	Move top of buffer to stack
Left-Arc	Create arc from stack top to second element; remove second
Right-Arc	Create arc from second element to stack top; remove stack top

7. Ethics & Information Extraction

7.1 Ethical Concerns in NLP

Issue	Description
Bias	Models inherit and amplify biases present in training data (gender, race, etc.)
Environmental Cost	Training large models requires huge compute resources and energy
Transparency	Neural models are "black boxes" -- difficult to explain decisions
Copyright	Training on copyrighted data raises legal questions
Liability	Who is responsible when AI generates harmful content?
Education Impact	AI text generation affects academic integrity

7.2 Best Practices

- **Datasheets for datasets:** document data collection, composition, intended use
 - **Model cards:** document model performance, limitations, biases, intended use
-

8. Information Extraction (Advanced)

8.1 Named Entity Recognition (NER)

IOB Tagging Scheme: - **B-TYPE:** Beginning of entity - **I-TYPE:** Inside (continuation of) entity - **O:** Outside any entity

Example: [Barack]_{B-PER} [Obama]_{I-PER} [visited]_O [Paris]_{B-LOC}

Approaches: CRF (Conditional Random Fields), Transformer-based sequence labeling

8.2 Entity Linking

Goal: Map entity mentions (strings) to knowledge base entries (things).

Pipeline: 1. **Candidate Generation:** - Dictionary method: lookup mention in alias table - Dense vector retrieval: encode mention and entities, find nearest 2. **Candidate Reranking (optional):** - Transformer model scores each candidate in context

"From strings to things" -- resolving that "Paris" in context refers to Paris, France vs. Paris, Texas.

8.3 Relation Extraction

Goal: Extract triples (subject, relation, object) from text.

Approach	Description	Pros/Cons
Co-occurrence	Two entities in same sentence	No actual relation type extracted
Rule-based	Patterns like "X, who starred in Y"	High precision, low recall
BERT-based	Entity markers + pooling for classification	Best performance; needs training data

BERT-based approach: Insert special markers around entities in input: [E1] Barack Obama [/E1] visited [E2] Paris [/E2] . Use the [E1] and [E2] representations for classification.

8.4 Coreference Resolution

Types of referring expressions: - **Anaphora:** refers to something mentioned earlier ("Barack Obama... **he**...") - **Cataphora:** refers to something mentioned later ("Before **he** left, Obama...") - **Exophora:** refers to something outside the text ("Look at **that**!")

Pipeline: 1. **Mention detection:** find pronouns, named entities, NPs 2. **Mention clustering:** determine which mentions refer to the same entity

Hobbs' Algorithm: syntactic tree-based method to find the noun phrase a pronoun refers to.

9. Large Language Models

9.1 Training Pipeline

1. **Pretraining:** Learn language on massive text corpora (causal LM objective)
2. **Instruction Tuning:** Fine-tune on instruction-following examples
3. **RLHF (Reinforcement Learning from Human Feedback):** Align model outputs with human preferences

9.2 Prompt Engineering

Strategy	Description	Example
Zero-shot	No examples provided	"Classify this as positive/negative: ..."
Few-shot	Provide examples in prompt	"Example 1: ... -> positive. Now classify: ..."
Chain-of-Thought (CoT)	Ask model to reason step-by-step	"Let's think step by step..."
Explicit instructions	Detailed task description	"You are a sentiment classifier. Output only..."

9.3 LLM Issues

Issue	Description
Factuality (Hallucination)	Models generate plausible but false information
Dangerous Generations	Harmful, biased, or toxic outputs
Test Set Leakage	Training data may contain benchmark test sets
Expense	Inference costs for large models are significant

9.4 Zero-Shot Evaluation Caveat (2024 Q3a)

Before claiming "zero-shot" performance, verify that: - The LLM's pretraining data did NOT include the test dataset - The prompt itself does not contain examples from the dataset - The model was not instruction-tuned on similar tasks

9.5 Computational Cost Ranking (2025 Q3a)

Logistic Regression < BERT-base < LLaMA-70B

- Logistic regression: lightweight linear classifier over sparse features
- BERT-base: ~110M parameter transformer
- LLaMA-70B: ~70B parameter transformer (600x larger than BERT)

9.6 Multilingual Considerations (2025 Q3c)

- **Logistic regression problem:** vocabulary is language-specific; tokens in new language are all unknown
- **BERT problem:** subword tokenizer trained on specific language(s); other languages split into meaningless character sequences
- **Solution:** Use multilingual models (mBERT, XLM-R), or collect/train on new language data

9.7 Subword Tokenization Pitfalls (2025 Q3d)

Brand name "BeStep" might tokenize as "Best" + "##ep", causing false positive sentiment. **Solution:** Replace brand names with neutral placeholders like "[BRAND]".

10. Exam Strategy & Past Paper Analysis

10.1 Exam Structure (Based on 2022-2025 Papers)

- **3 compulsory questions**, ~20 marks each (60 total in recent years)
- **Duration:** 1h 30m + 30m additional time
- **Open book** -- crib sheet provided
- **Mix of:** calculation, pseudo-code, explanation/discussion, worked examples

10.2 Recurring Question Types

Type	Frequency	Topics
Similarity calculation	Every year	Jaccard, Dice, Overlap, Cosine, TF-IDF
N-gram language model	Every year	Build model, generate text (greedy/beam), smoothing
BPE tokenization	2024, 2025	Apply merge rules, discuss vocab size
Classification metrics	Every year	Confusion matrix, P/R/F1, macro/micro, anomalies
Embeddings discussion	2022	Static vs contextual, polysemy, word2vec
Applied NLP scenario	Every year	Real-world system design, error analysis
BERT vs BoW	2022, 2025	How to incorporate features, transfer learning
Ethical/practical issues	2024, 2025	Bias, cost, multilingual, dataset quality

10.3 Mark Distribution Strategy

Time	Marks	Approach
First 25 min	Q1 (~20 marks)	Often calculation-heavy; do carefully
Next 25 min	Q2 (~20 marks)	Often language model / generation
Next 25 min	Q3 (~20 marks)	Often discussion / applied scenario
Final 15 min	Review	Check calculations, add detail to discussions

10.4 High-Scoring Answer Patterns

From sample answers, examiners reward:

1. **Domain-specific examples** -- not generic ones. "In patent classification, the word 'light' can be noun (F: lighting) or adjective (D: lightweight fabric)."
2. **Explicit working** -- show all counts, intermediate steps, and final answer.
3. **Addressing all parts** -- if asked for "three problems," give exactly three (or more).

4. **Connecting concepts** -- "This is overfitting because train performance improved but test performance degraded."

5. **Mentioning trade-offs** -- "While BERT captures context better, it is computationally more expensive."

10.5 Common Mistakes to Avoid

- Confusing **similarity** and **distance** (they are inversely related)
- Forgetting that **Sorensen-Dice uses set cardinality, not token count** (duplicates don't count)
- Confusing **types** (unique tokens) vs **tokens** (total occurrences)
- Not converting **similarity to distance** before checking triangle inequality
- Using **wrong context** in n-gram generation (forgetting newly generated tokens are part of context)
- Reporting **Macro** metrics for binary classification (just use P/R/F1)
- **F1 must always be between precision and recall** -- if not, something is wrong

10.6 Topic-to-Question Mapping

Lecture	Key Exam Topics
L1: Tokens & Similarity	Tokenization, set similarity, regex, n-grams, metric properties
L2: Vectors & Clustering	TF-IDF calculation, cosine similarity, K-means, silhouette
L3: Language Modelling	N-gram models, smoothing, perplexity, generation strategies
L4: Classification	Metrics (P/R/F1/AUROC), overfitting, cross-validation, data issues
L5: Embeddings	Static vs contextual, attention, BPE, BERT vs GPT, transfer learning
L6: POS & Parsing	HMM, Viterbi, constituency/dependency parsing, transition-based
L7: Ethics	Bias, environmental cost, transparency, best practices
L8: Info Extraction	NER (IOB), entity linking, relation extraction, coreference
L9: LLMs	Training pipeline, prompt engineering, RLHF, issues

11. Crib Sheet Reference (All Formulas)

Similarity Measures

Overlap Count: $|X \cap Y|$
 Overlap Coefficient: $|X \cap Y| / \min(|X|, |Y|)$
 Sorensen-Dice: $2|X \cap Y| / (|X| + |Y|)$
 Jaccard: $|X \cap Y| / |X \cup Y|$

Distance Measures

Euclidean: $d(x,y) = \sqrt{\sum_i (x_i - y_i)^2}$
 Cosine: $\cos(\theta) = (d_1 \cdot d_2) / (||d_1|| * ||d_2||)$

TF-IDF

Log TF: $1 + \log_{10}(tf)$ if $tf > 0$, else 0
 IDF: $\log(N / df_t)$
 TF-IDF: $(1 + \log(tf)) * \log(N / df_t)$

Language Model

Add-k: $(count + k) / (N + k * |V|)$
 Interpolation: $l_1 * P(t_i | t_{i-1}, t_{i-2}) + l_2 * P(t_i | t_{i-1}) + l_3 * P(t_i)$
 Jelinek-Mercer: $\lambda * P(t | doc) + (1 - \lambda) * P(t | collection)$
 Perplexity: $(\prod P(w_i))^{(-1/n)} = 2^{CrossEntropy}$
 Cross-Entropy: $-(1/n) * \sum(\log_2(P(w_i)))$
 Entropy: $-\sum(P(x) * \log_2(P(x)))$

Classification Metrics

Accuracy: $(TP+TN) / (TP+FP+TN+FN)$
 Precision: $TP / (TP+FP)$
 Recall: $TP / (TP+FN)$
 F1: $2 * TP / (2 * TP + FP + FN) = 2 * P * R / (P + R)$
 F-beta: $(1 + B^2) * P * R / (B^2 * P + R)$

Self-Attention

```

Q = Input * W_Q
K = Input * W_K
V = Input * W_V
Attention(Q,K,V) = softmax(Q*K^T / sqrt(d_k)) * V

```

HMM

```

Transitions: P(y_i | y_{i-1})
Emissions:   P(x_i | y_i)

```

Silhouette Score

```

silhouette(i) = 1 - cohesion/separation    if cohesion < separation
                0                          if cohesion = separation
                separation/cohesion - 1     if cohesion > separation

```

Metric Properties

1. Identity: $d(X,X) = 0$
2. Positivity: $d(X,Y) > 0$ for $X \neq Y$
3. Symmetry: $d(X,Y) = d(Y,X)$
4. Triangle Inequality: $d(X,Z) \leq d(X,Y) + d(Y,Z)$

Python String Operations

```

a + b                # Concatenate
text[i:j]            # Substring from i to j
text[:i]             # Remove final i characters
.upper() / .lower()  # Case conversion
.isupper() / .islower() # Case check
.strip() / .lstrip() / .rstrip() # Remove whitespace
.replace(a, b)       # Replace substring
.split(a)            # Split by delimiter (default: whitespace)
.removeprefix() / .removesuffix() # Remove prefix/suffix
.startswith() / .endswith() # Check prefix/suffix
.find() / .count()   # Find/count substrings
a.join(l)            # Join list with delimiter

```

Quick Revision Checklist

- ☐ Can I calculate Jaccard, Dice, Overlap from two tokenized documents?
 - ☐ Can I build unigram/bigram/trigram tables and generate text with greedy/beam search?
 - ☐ Can I apply add-k smoothing to an n-gram model?
 - ☐ Can I calculate Precision, Recall, F1 from a confusion matrix?
 - ☐ Can I identify overfitting/underfitting from train vs test metrics?
 - ☐ Can I trace through BPE merge rules on a given input?
 - ☐ Can I explain the self-attention formula (Q, K, V)?
 - ☐ Can I explain the difference between BERT (encoder) and GPT (decoder)?
 - ☐ Can I describe the NER pipeline (IOB tagging, entity linking)?
 - ☐ Can I discuss ethical issues with concrete examples?
 - ☐ Can I explain why static embeddings fail on polysemy?
 - ☐ Can I describe K-means clustering steps and evaluation (silhouette)?
 - ☐ Can I explain transfer learning (pretrain + fine-tune)?
 - ☐ Can I design a classification experiment (data split, metrics, statistical significance)?
-

Total: ~900 lines | Covers all 10 lectures + past papers (2022-2025) + complete crib sheet formulas Last updated: March 2026